

smalltalk-dev-plugin の紹介

Pharo Smalltalk 向け AI 駆動開発ツールキット

梅澤 真史

<https://github.com/mumez/smalltalk-dev-plugin>

smalltalk-dev-plugin とは？

AI コーディングエージェントと Pharo Smalltalk をつなぐ包括的なツールキットです。

AI コーディングエージェントをそのまま Smalltalk に使ってもうまくいきません：

- **Pharo の知識がない** — AI は環境の操作方法を知らない
- **プロジェクト構造が不明** — Tonel やパッケージ構成を知らない
- **テスト・デバッグができない** — テスト実行やエラー診断の仕方がわからない
- **ライブラリへのアクセスができない** — 既存のクラスやメソッドを参照できない
- **コーディングスタイルが悪い** — Smalltalk のイディオムや慣習を知らない

これらは Smalltalk で AI 支援を活用したい開発者にとって大きな障壁でした。

AI に Smalltalk 開発の完全なスキルセットを与えました：

課題	解決策
環境との対話	Pharo とやりとりするための MCP ツール群
プロジェクト構造	プロジェクトテンプレート付き <code>/st-setup-project</code>
テスト・デバッグ	<code>/st-test</code> 、 <code>/st-eval</code> 、デバッガースキル
ライブラリ参照	ファインダースキル、コード検索ツール
コード品質	バリデーター、リンター、コメントースキル

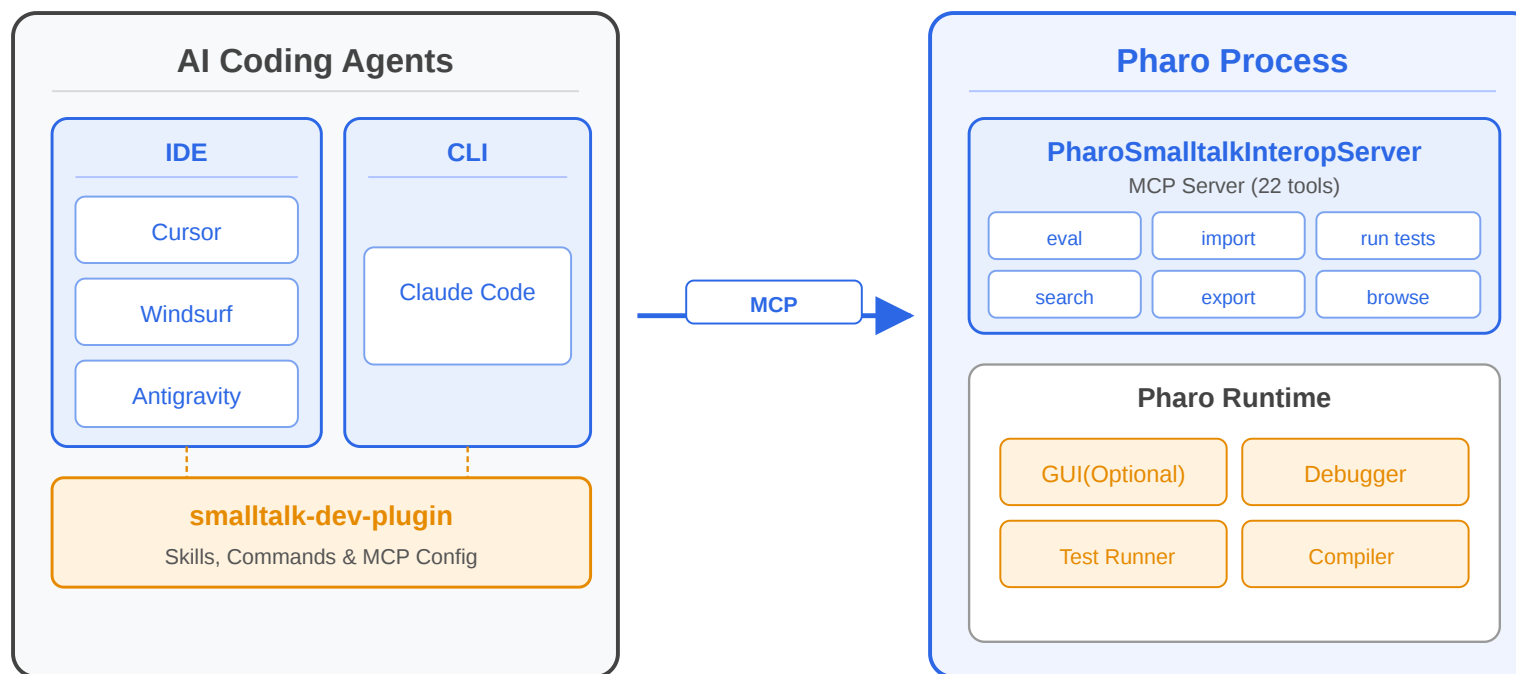
結果: AI が信頼できる Smalltalk 開発パートナーになります！

「AIによる編集を唯一の真実の源として扱う」

AI が *Tonel* ファイルを編集し、*Pharo* にインポートして、テストするという流れを繰り返す。

概要

- 9 個のスラッシュコマンド
- 5 種類の専門 AI スキル
- 22 種の *Pharo* との連携用 MCP ツール
- 自動コード品質チェックとコメント生成



- 既存の AI エージェントが MCP 経由で Pharo と通信 — カスタム IDE 不要
- ヘッドレス Pharo にも対応 — CI やリモートエージェント (Devin など) でも利用可能

エージェント	サポートレベル
Claude Code	フルサポート（メイン）
Cursor	スクリプトによる簡易セットアップ
Windsurf	スクリプトによる簡易セットアップ
Antigravity	スクリプトによる簡易セットアップ
GitHub Copilot CLI	スクリプトによる簡易セットアップ
OpenCode	スクリプトによる簡易セットアップ

本プレゼンテーションでは Claude Code での利用を中心に紹介します。

インストール

インストール (1) — プラグインのインストール

```
# GitHub からマーケットプレイスを追加
```

```
claude plugin marketplace add mumez/smalltalk-dev-plugin
```

```
# プラグインをインストール
```

```
claude plugin install smalltalk-dev
```

オプションA: Docker (簡単)

[smalltalk-interop-docker](#) を使って設定済み Pharo イメージを起動：

```
docker compose up -d
```

オプションB: ローカルでPharoを準備

PharoSmalltalkInteropServerを手動でインストールして起動：

```
Metacello new
  baseline: 'PharoSmalltalkInteropServer';
  repository: 'github://mumez/PharoSmalltalkInteropServer:main/src';
  load.

SisServer current start.
```

各エージェント向けのセットアップスクリプトが用意されています：

```
./extra/setup-cursor.sh [target-directory]
./extra/setup-windsurf.sh [target-directory]
./extra/setup-antigravity.sh [target-directory]
./extra/setup-copilot.sh [target-directory]    # GitHub Copilot CLI
./extra/setup-opencode.sh [target-directory]    # OpenCode
```

制限事項

- エージェントによって機能が異なる場合があります
- Pharo 側のセットアップは Claude Code と同じです

基本的な使い方

空のディレクトリまたは既存プロジェクトのディレクトリで `claude` を実行します。

- Iceberg リポジトリのディレクトリを指定することもできます
- ただし競合を避けるため、リポジトリを別のディレクトリにクローンしてそちらを使うことを推奨します

`/st-buddy` は適切なツールに誘導してくれる親しみやすいアシスタントです。

やりたいことを言葉で説明するだけです：

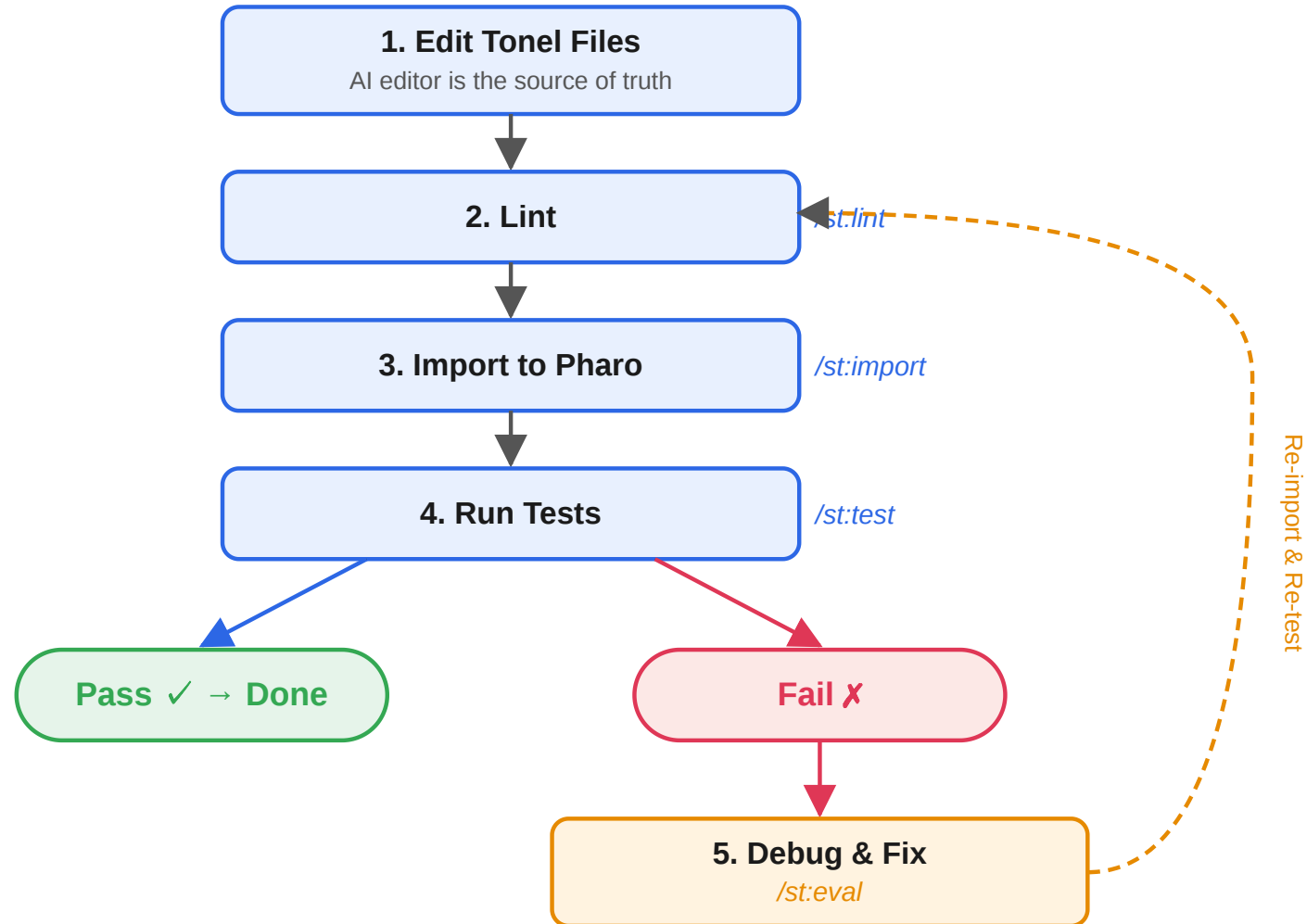
```
/st-buddy nameとageを持つPersonクラスを作成したい
```

アシスタントが自動的に以下を行います：

1. 適切なスキルの読み込み
2. Tonel ファイルの作成
3. リント、インポート、テストの支援

意図	読み込まれるスキル
「作成したい」「追加したい」	smalltalk-developer
「エラーが出た」「テストが失敗した」	smalltalk-debugger
「使い方は？」	smalltalk-usage-finder
「誰が実装している？」	smalltalk-implementation-finder

どのツールを使えばよいか覚える必要はありません — `/st-buddy` に話しかけるだけです。



コンポーネント

smalltalk-developer

コアとなる開発スキル。Tonel フォーマット、パッケージ構造、「編集 → リント → インポート → テスト」のワークフロー全体を把握しています。

smalltalk-debugger

体系的なデバッグの専門家。エラーを診断し、`/st-eval` を使って段階的にコードを実行し、修正方法をガイドします。

smalltalk-usage-finder

クラスやメソッドの使われ方を調査します。使用例を見つけ、利用パターンを分析し、未知の API の理解を助けます。

smalltalk-implementation-finder

クラス階層を横断してメソッドの実装を検索します。コーディングイディオムの学習やリファクタリングの影響評価に役立ちます。

smalltalk-commenter

ドキュメント不足のクラスに対して **CRC スタイルのクラスコメント** を自動提案します。フック経由でファイル編集後にトリガーされます。機械的にではなく、ドキュメントが最も必要なクラスを優先します。

コマンド	目的
<code>/st-buddy</code>	フレンドリーアシスタント（メインエントリー）
<code>/st-init</code>	開発セッションの初期化
<code>/st-setup-project</code>	プロジェクトボイラープレートの作成
<code>/st-eval</code>	Smalltalk 式の実行
<code>/st-import</code>	Tonel パッケージを Pharo にインポート
<code>/st-export</code>	Pharo からパッケージをエクスポート
<code>/st-test</code>	SUnit テストの実行
<code>/st-lint</code>	コード品質チェック
<code>/st-validate</code>	Tonel 構文の検証

2 つの MCP サーバーが連携の裏側を支えています：

pharo-interop (22 ツール)

Pharo との通信 — eval、インポート/エクスポート、テスト、コードナビゲーションなど。

smalltalk-validator (5 ツール)

Tonel の検証とリント。

注意: これらを直接操作する必要はほとんどありません。

オーケストレーションは `/st-buddy` が担当します。

ライブデモ

空のディレクトリから Smalltalk 初心者として始めるシンプルなユースケース。

```
/st-buddy 私はSmalltalkの初心者です。  
異なる通貨単位間で算術演算を行えるMoneyクラスを作成したいです。  
プロジェクトの初期設定から手助けして。
```



```
Claude Code — Example 1: Money Class

e-money/src/SampleMoney/ExchangeRateProvider.class.st")
└─ success: true,
  file_path: "/home/una/git/st-dev-sample-money/src/SampleMoney/ExchangeRateProvider.class.st",
  +11 lines (ctrl+o to expand)

plugin:smalltalk-dev:smalltalk-validator - lint_tonel_smalltalk_from_file (MCP)(file_path:
"/home/una/git/st-dev-sample-money/src/
SampleMoney-Tests/MoneyTest.class.st")
└─ success: true,
  file_path: "/home/una/git/st-dev-sample-money/src/SampleMoney-Tests/MoneyTest.class.st",
  +5 lines (ctrl+o to expand)

plugin:smalltalk-dev:smalltalk-validator - lint_tonel_smalltalk_from_file (MCP)(file_path: "/home/una/git/st-dev-sampl
e-money/src/BaselineOfSampleMoney/Basel
ineOfSampleMoney.class.st")
└─ success: true,
  file_path: "/home/una/git/st-dev-sample-money/src/BaselineOfSampleMoney/BaselineOfSampleMoney.class.st",
  +9 lines (ctrl+o to expand)

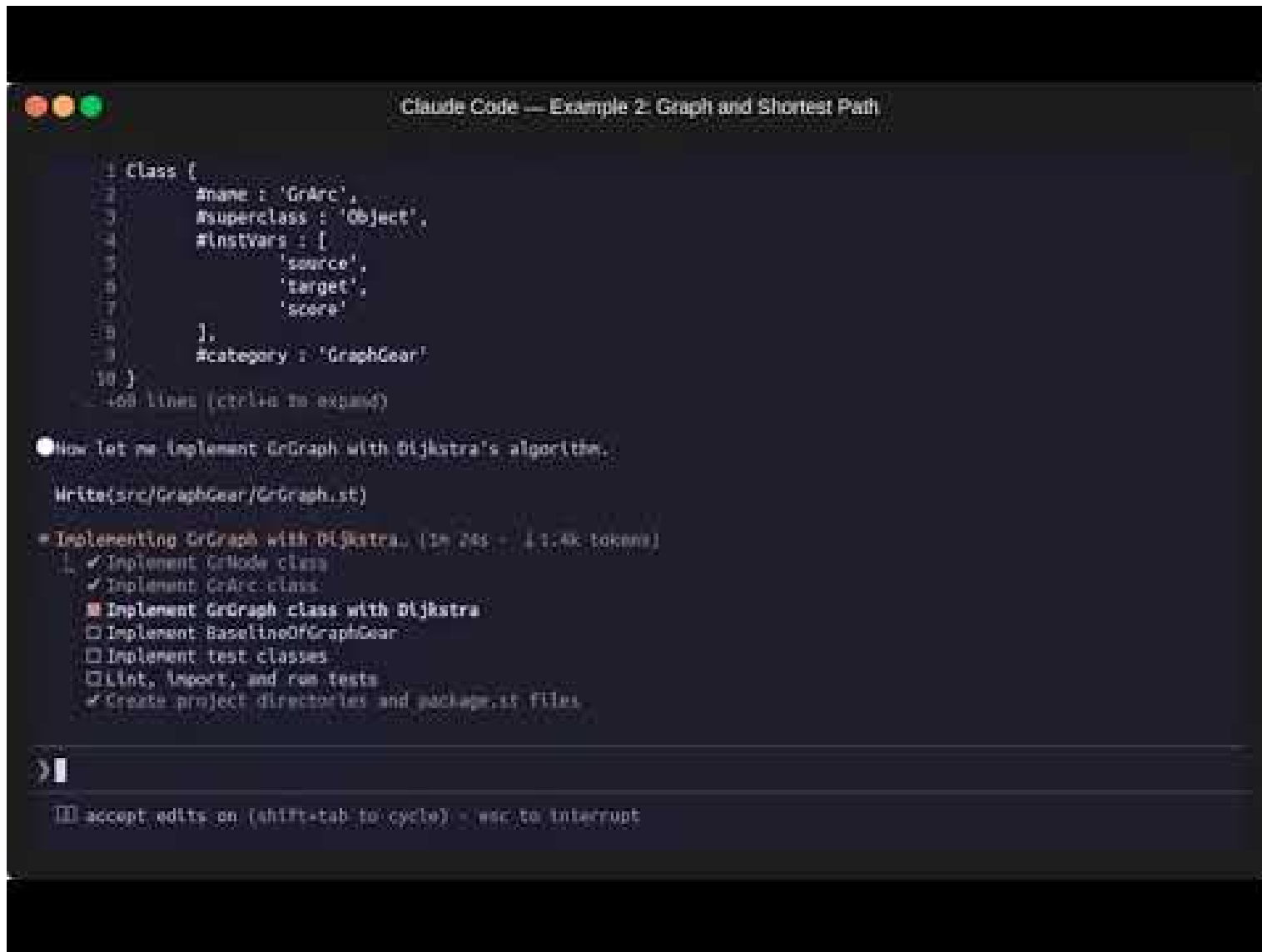
- Scarrying... (in 28s - 11.2k tokens)

> |
[?] accept edits on (shift+tab to cycle) - esc to interrupt
```

- 動画
- 生成されたソース
- デモ

Smalltalk に慣れたユーザー向けのより複雑なユースケース。

/st-buddy 有向グラフを表現するためにGrNodeとGrArcを作成し、最短経路問題を解きたいと考えています。
ノードにはnameが、アークにはscoreがあります。
GraphGearプロジェクトとして始めましょう。



```
Claude Code — Example 2: Graph and Shortest Path

1 class [
2     #name : 'GrArc',
3     #superclass : 'Object',
4     #instance_vars : [
5         'source',
6         'target',
7         'score'
8     ],
9     #category : 'GraphGear'
10 ]
11 }
12 466 lines (ctrl+u to expand)

Now let me implement GrGraph with Dijkstra's algorithm.

Write(src/GraphGear/GrGraph.st)

= Implementing GrGraph with Dijkstra. (1m 24s - 11.4k tokens)
  [
    ✓ Implement GrNode class
    ✓ Implement GrArc class
    ■ Implement GrGraph class with Dijkstra
    □ Implement BaselineOfGraphGear
    □ Implement test classes
    □ Lint, import, and run tests
    ✓ Create project directories and package.st files
  ]

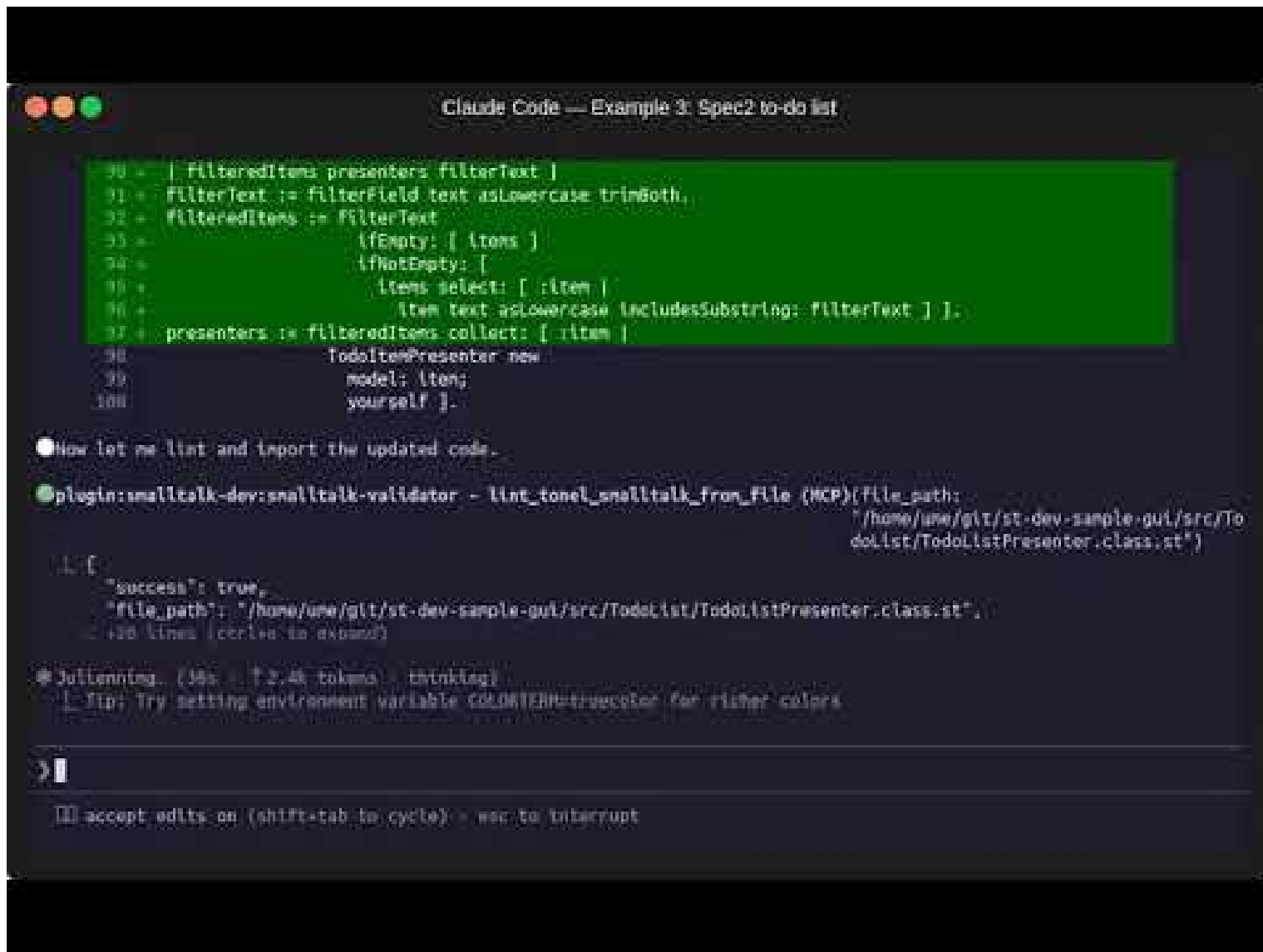
> |

accept edits on (shift+tab to cycle) - esc to interrupt
```

- 動画
- 生成されたソース
- デモ

Spec2 を使った GUI アプリケーションの構築。

/st-buddy Spec2フレームワークを使って、シンプルなToDoリストを作成したい。
各項目にチェックボックスと入力フィールドを配置し、
下部に「追加」と「削除」のボタンを配置する。
削除できるのは、チェックされている項目のみとする。着手して。



The screenshot shows a Claude Code window titled "Claude Code — Example 3: Spec2 to-do list". The code editor contains the following Smalltalk code:

```
30 + | filteredItems presenters filterText |
31 + filterText := filterField text asLowercase trimBoth.
32 + filteredItems := filterText
33 +     ifEmpty: [ items ]
34 +     ifNotEmpty: [
35 +         items select: [ :item |
36 +             item text asLowercase includesSubstring: filterText ] ].
37 + presenters := filteredItems collect: [ :item |
38 +     TodoItemPresenter new
39 +         model: item;
40 +         yourself ].
```

Below the code editor, the execution results are shown:

- Now let me list and import the updated code.
- plugin:smalltalk-dev:smalltalk-validator - lint_tonel_smalltalk_from_file (MCP){file_path: "/home/une/git/st-dev-sample-gui/src/TodoList/TodoListPresenter.class.st"}
- └─ {
 "success": true,
 "file_path": "/home/une/git/st-dev-sample-gui/src/TodoList/TodoListPresenter.class.st",
 "lines": 40 (ctrl+q to expand)}
- Julianneq. (38s - 12.4k tokens - thinking)
└─ Tip: Try setting environment variable COLORTERM=truecolor for richer colors
-
- accept edits on (shift+tab to cycle) - esc to interrupt

- 動画
- 生成されたソース
- デモ

ケーススタディ

ケーススタディ — 実際のプロジェクト

smalltalk-dev-plugin を使って作成された実際のプロジェクトを紹介します。

どちらも **90% 以上が Claude Code + smalltalk-dev-plugin で作成**されました。

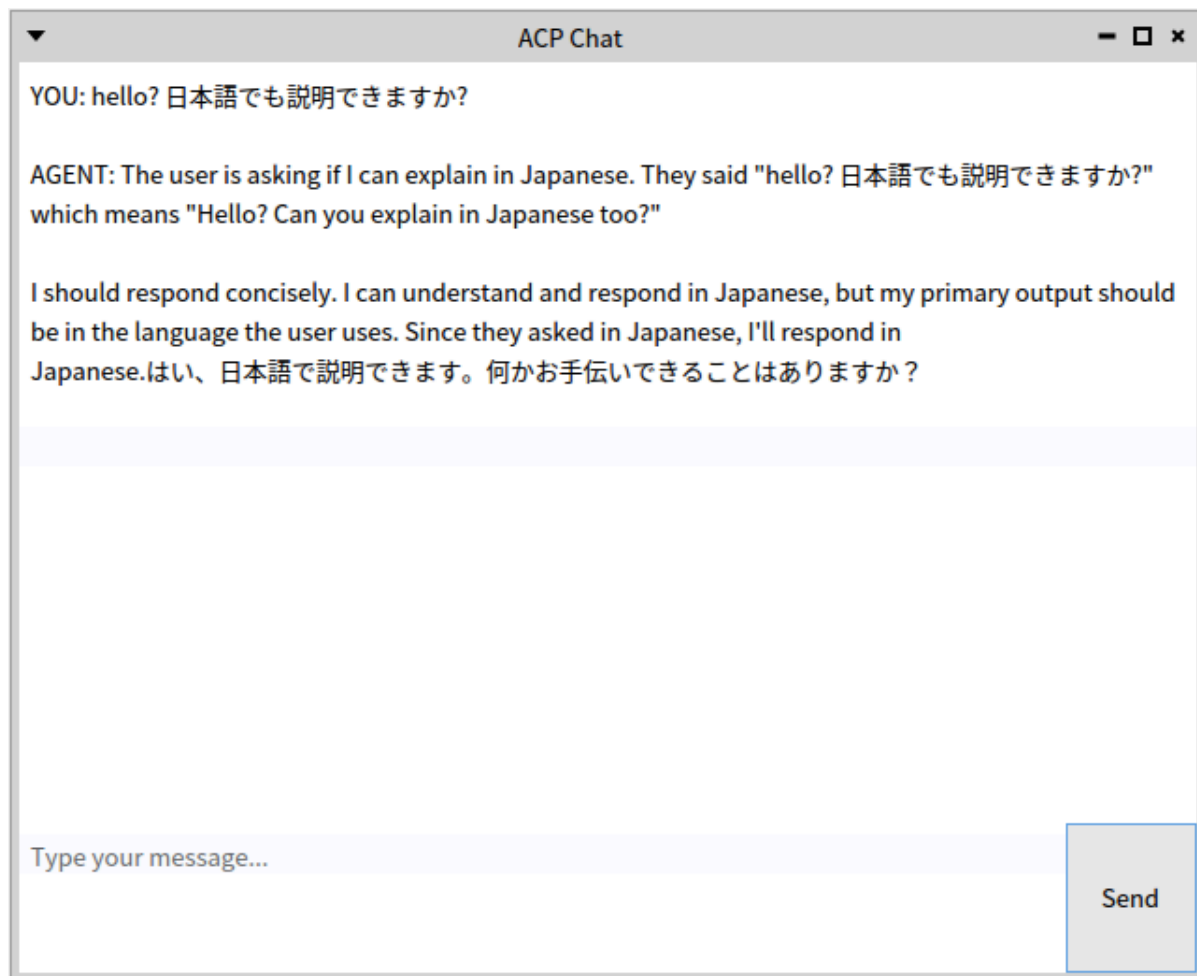
ACP プロトコルで Pharo 側から AI エージェントに接続するためのライブラリ。

接続先: Gemini CLI、Claude Code、OpenCode、Copilot CLI, Codex CLI など

<https://github.com/mumez/pharo-acp>

Pharo 向けシンプルな AI Chat GUI。内部で pharo-acp を使用。

<https://github.com/mumetz/pharo-acp-chat-ui>



smalltalk-dev-plugin は AI アシスタントによる Pharo Smalltalk 開発を実用的かつ生産的にします。

`/st-buddy` と入力して、開発を始めましょう。

フィードバックとコントリビューションをお待ちしています！

<https://github.com/mumez/smalltalk-dev-plugin>